

Operation fsoc

Case Study of a Defense-First Security Engineering Project

Gurmann Basran

Prepared 2026-04-16. Revised 2026-06-15 (rev. 3). Updated as the project progresses.

1 What this is

Operation fsoc is the name I gave to the multi-phase security engineering project I have been running out of my own homelab since August 2025. It started as a homelab upgrade and grew into something more useful: an operational infrastructure for an emerging cybersecurity professional, built from scratch on a student budget, designed so that every phase teaches me a domain of the field while also locking down a real system I use every day. From early on I have run it as a systems-engineering effort rather than a series of one-off tweaks: a charter, traceable requirements, an architecture and interface baseline, a structured audit, and a verification discipline, applied to a personal homelab with the same rigor those tools earn on a fielded system.

This document is the public face of the project. It is a case study, not a runbook. Specific products, version numbers, exact architectural details, IP addresses, hostnames, domain names, rule IDs, script internals, and credential paths have all been deliberately kept out of this writeup. So have narrow category-level descriptions that would narrow a tool choice to one or two likely candidates. The reason is simple: publishing how I think is useful to another engineer, a hiring manager, or an evaluation reviewer; publishing the shape of my specific stack would primarily serve an attacker looking for known-version vulnerabilities. I have kept the methodology and the reasoning and stripped almost everything else.

The short version of what I did. I built layered defensive infrastructure across two physical machines, documented a per-phase threat model, ran a structured adversarial audit of the whole result, closed every fix-required finding with a trace from the finding ID to the commit that fixed it, ran a second hardening pass on top of that, and wrote a set of production monitoring and auto-healing scripts that keep the whole thing running while I am at class. The most recent milestone took the machine I administer all of this from, which used to be the weakest-reasoned part of the system, and rebuilt it under the same discipline as the infrastructure. The work since then has been less about adding new systems and more about proving the ones I already have: I finished the cutover to certificate-only fleet authentication, added an operational-health dimension to the monitoring after an outage the security audit was never built to catch, retired an unused service along with a redundant external entry path, and verified that the backups actually restore rather than assuming they would. The companion documents in this repository describe the methodology, the threat model, the script design patterns, the integration lessons, and that workstation migration, in each case at a level of abstraction that a reader can learn from without extracting my configuration.

2 Why I built it

Mr. Robot shaped how I think about operating online: compartmentalize everything, trust nothing you cannot verify, and make sure the tools you use are things you actually understand. The project

name is a nod to the fsociety collective in the show. The substance, though, is not about mimicking a TV series; it is about building the things a security professional actually needs to understand, in a setting where getting it wrong has consequences I have to live with.

The other reason is practical. I am a Computer Science student and I plan to work in security. The certifications I want (CompTIA Security+, then OSCP) are on my roadmap but I have not earned them yet. The fastest path I know to turn that gap into credibility is to build something real, document it rigorously, and make the documentation legible to someone evaluating my work. That is what this repository is.

3 Architecture, at an abstract level

The system is physically compartmentalized across two machines. One is always on and runs the infrastructure services. The other is on-demand and runs the operations work. The rationale is that if the operations side were ever compromised, the compromise would not bleed into the secrets store, the password vault, or the daily network. The two trust zones share nothing but a switch port.

The logical network is segmented into multiple trust zones. There is a zone for everyday home use, a zone for self-hosted services and their admin plane, a zone for anonymity-focused activity, and a zone for offensive learning. Each zone has its own policy. The offensive zone can reach the internet for training labs but cannot reach any of the other zones; a mistake in a CTF box cannot become a mistake on my password vault.

There is a third role in the picture, and for most of the project it was the part I had thought about the least: the workstation I actually sit at to administer everything. A fortress run from a soft machine is not a fortress; the workstation holds operator access, so it belongs inside the threat model rather than in front of it. The most recent milestone rebuilt that machine, and it gets its own companion document because the reasoning is worth more than the parts list.

I am not including a more detailed network diagram or zone map in this public writeup. The exact segmentation, inter-zone rule structure, and service placement are the kind of detail that is useful primarily to someone trying to target the system, and they are not required to evaluate whether the design is reasonable.

4 Tools

I am not listing the tools I used, at any level of specificity. A public category-by-category breakdown is still narrow enough that a reader familiar with the field can often reduce each slot to one or two likely products, and the combination of those reductions is a plausible stack fingerprint that I would rather not put on the internet next to my real name.

What is happening inside the box is the usual stuff. There is packet filtering at the perimeter and between zones, there is detection that looks at traffic and at hosts, there is encrypted remote access for devices I own, there is identity-gated access to anything I care about, there is a place for secrets that is not on any individual machine, there is a layered path for name resolution, there is a logging

backbone that the detection layer watches, and there is a fleet-wide authentication scheme that hands each device a short-lived signed identity instead of a long-lived static key it would otherwise carry forever. Each slot is filled by an open-source or community-maintained tool that was chosen against at least one alternative, with the reasoning documented in my private planning directory.

If you are reading this to evaluate whether the design is reasonable, what matters is the shape of the whole, not which specific product fills each slot. If you are reading this because you build something similar and you are curious which tool I picked for a given layer, I am happy to talk in a non-public channel.

5 Defensive bias of the roadmap

The roadmap is deliberately weighted toward defensive work. The majority of the phases harden something, add detection somewhere, reduce the attack surface of some component, or improve the resilience of the system. A smaller set are hybrid: writing detection requires understanding the attack it is meant to detect, so those phases have a defensive purpose even when they involve reading offensive material. A smaller still set are offensive learning phases: recon, web exploitation, binary exploitation, anonymity operations, physical security assessment, and an advanced track aimed at OSCP preparation.

This bias is intentional. Defense-first does not mean offense-last; it means offense *compartmentalized*. The offensive phases run inside a segregated zone with no network path to the rest of the infrastructure. If something I am practicing against in a CTF box ever gets out of the CTF box, the blast radius stops at one zone.

6 A systems-engineering approach

I run this project the way a systems-engineering effort is run, not as a loose pile of notes. There is a project charter that fixes the scope, the constraints, and the operating principles. There is a requirements specification in which every requirement carries a stable identifier, and a requirements traceability matrix that ties each requirement to the design that satisfies it and the verification that proves it. The architecture and the interfaces between components are captured as their own version-controlled documents; significant design decisions are recorded as dated decision records that name the alternatives I rejected and why; and an as-built description tracks the live system and is reconciled against reality rather than allowed to drift. Each phase sits inside that baseline with its own research, decision log, pre-flight checklist, execution plan, verification checklist, and post-mortem note. The planning is written before the configuration is, and all of it is configuration-managed in version control rather than carried in my head.

This is deliberate, because the cost of a bad change on a live system is significant. Early in the project I sequenced a dangerous reconfiguration before the safety net was in place and nearly caused a multi-hour internet outage. The refined phase-ordering principle that came out of that incident is in the companion document on hardening methodology. The short version: safe changes first, then reversible changes, then dangerous changes with a dead-man SSH session open, then cosmetic cleanup. Every phase is gate-checked; an unchecked box blocks the phase from starting.

The work is grouped into milestones, and the same loop runs inside each one. The infrastructure build came first. Then a structured audit went over the whole result as an adversary would, across several security domains, and produced close to a hundred findings, each with a stable ID and a tag for the action it demanded. A remediation milestone closed the findings that needed a change. A hardening milestone layered a second set of requirements on top of the closed findings. The workstation migration and a recovery-insurance milestone are the most recent. The loop has now run enough times that I trust it, which is its own kind of result.

Recently I have automated the execution of that process without loosening its controls, the way a delivery pipeline automates a release while still gating the risky steps behind a human. An orchestration layer classifies each change by its risk and reversibility and applies the matching level of governance: low-risk reversible work proceeds on its own, anything that touches the administrative plane or cannot be cleanly undone is held for an explicit approval, and a defined set of high-consequence change classes can never proceed without me. Every run leaves a configuration-managed record of what it changed and why, which is the same traceability the rest of the methodology already demands, and a coordination layer lets parallel workstreams run without colliding by giving each an exclusive claim on what it is touching. The automation enforces the discipline consistently; it does not make the engineering decisions. The requirements, the threat model, the architecture, and the verification criteria are mine, and the orchestration exists only to apply them the same way every time, with every consequential step gated to a human and recorded. The reason it is worth building is practical: I do this in fragmented hours around a degree, and a governed process that survives being interrupted is the only kind that actually gets finished.

7 Status

I am not publishing the granular phase status because the phase IDs themselves would help someone trying to reverse-engineer what I have and have not hardened yet. At a high level: the foundational network work is complete; a structured multi-domain audit has been run and every fix-required finding closed with traceability; a remediation pass and a second hardening pass are complete; fleet authentication has finished its cutover to short-lived signed certificates only, with the last long-lived static key removed and a self-checking gate that refuses to let one creep back; the operator workstation has been migrated onto a native, compartmentalized, hardened platform whose self-healing layer is now verified; a browser-based console for reaching that machine remotely is already live, and a fuller terminal-native command center is being built on top of it; a recovery-insurance milestone has proven the backups actually restore, confirmed that the backup encryption key is escrowed off the fleet, and put a staleness alarm on the backups themselves; and an operational-health dimension has been added to the monitoring after a class of outage the security audit was structurally blind to. The next two security milestones are a centralized off-host logging spine and an egress-VPN privacy layer, and research on both is done; the offensive learning phases are planned and have not started. There is also an external deadline I am building toward: a planned physical relocation of the hardware to a new site, behind a new upstream provider, and because a move is a forced full restore it doubles as the real-world validation of every recovery path above.

8 Companion documents

The rest of the case study is split across six documents. Read them in order.

1. 2-ThreatModel.pdf, which sets up what I defend against, what I explicitly do not, and the assume-breach scenarios I gate every critical change on.
2. 3-HardeningMethodology.pdf, which describes the finding-with-traceability audit loop, how it scaled to a fleet-wide review, and the rule for how I sequence changes.
3. 4-ScriptsShowcase.pdf, which describes the production monitoring automation at the design-goal level, without code and without layer-specific mechanism.
4. 5-LessonsLearned.pdf, which collects the integration traps I hit, generalized to the class-of-trap level.
5. 6-Security.pdf, which states the redaction posture of this repository and explains how to tell me if you find a leak.
6. 7-WorkstationMigration.pdf, which describes bringing the operator workstation under the same threat model as the infrastructure, and the residual risk it does and does not close.

If you are reading to evaluate my work, I would start with 2-ThreatModel and 5-LessonsLearned. They are the two documents that most directly show how I think.